LLD Practice Platform

2-Day Engineering Assignment

The challenge

Design and build a small practice experience that helps a learner practice Low-Level Design, submit a solution, and receive useful, explainable feedback.

1. Background

LLD practice is often easy to start but difficult to evaluate. A learner can design a Parking Lot, Elevator, Vending Machine, or similar problem and still be unsure whether the responsibilities, abstractions, relationships, and trade-offs are actually good.

We want to explore a focused product that helps learners practice LLD repeatedly and understand where their design can improve.

2. Your Task

Research the problem briefly, define a focused MVP, and build a working prototype of an LLD practice platform. Your solution should concentrate on the learner journey rather than on building an LMS or a large assessment system.

A useful starting journey is:

Practice loop

Choose problem → Think / design → Submit → Get feedback → Review → Try again

3. What Your MVP Should Demonstrate

Area	Expected outcome
Problem	A small set of LLD problems with clear requirements and enough context to attempt them.
Practice	A learner can start an attempt and work on a solution in a form you choose (for example text, code, diagram, or a combination).
Submission	The learner can submit a solution and see its status.
Feedback	The platform provides useful feedback on the design. AI may be used where reasoning is helpful.
History	The learner can see previous attempts so the product supports improvement, not just one-time solving.
Core design	The important domain behaviour is represented with clear classes/interfaces and responsibilities.

4. The Main Design Questions

Your research and implementation should help answer these questions:

- What does a learner actually need to provide for an LLD practice attempt to be meaningful?
- What makes feedback useful when there can be more than one valid LLD solution?
- Which parts of evaluation should be deterministic, and which parts benefit from an LLM?
- How would your design accommodate another evaluation approach or another submission format later?
- What should happen if evaluation takes time or fails? Keep this practical; do not turn the assignment into a distributed-systems project.

5. Scope Boundary

This is primarily an LLD/domain-design exercise. Do not spend the majority of your time on Kubernetes, microservices, multi-region deployment, sharding, CDN design, or other large-scale HLD concerns. A simple monolith is completely acceptable.

Important distinction

LLD focus: classes, objects, responsibilities, interfaces, behaviour, relationships, patterns, extensibility, and code-level decisions. HLD is only a light consideration when explaining how your prototype would handle more users or slower AI evaluation.

6. AI Usage

You are encouraged to use ChatGPT, Claude, Gemini, Cursor, Copilot, or similar tools. We are evaluating how well you use AI together with your own judgement—not whether you avoid AI.

Include a short AI_USAGE.md explaining 3–5 meaningful AI-assisted decisions: what the AI suggested, what you accepted or rejected, and why.

7. What to Submit

Deliverable	What we expect
Research note	1–2 pages: learner problem, a few existing approaches/tools researched, key gaps, and your product direction.
Design note	A concise explanation of your MVP, user flow, important classes/interfaces, evaluation approach, and key trade-offs.
Working prototype	A demonstrable end-to-end practice flow from problem selection to feedback and attempt history.
Tests	Tests for important behaviour and at least a few failure/edge cases.
README + AI_USAGE.md	How to run the project, key decisions, limitations, and meaningful AI usage.

8. Time & Evaluation

Time: **2 days**. We expect a focused prototype, not a production-ready system.

Evaluation area	Weight
Problem understanding & research	15%
Product thinking / creativity	15%
LLD / domain design	25%
Evaluation & feedback approach	15%
Extensibility & engineering judgement	10%
Implementation quality	10%
Testing & reliability	5%
AI usage	5%

9. Submission

Please submit your completed requirements via this Google Form: https://docs.google.com/forms/d/e/1FAIpQLSfoHirkkjOeAYvzWksEVXSEQN5rEeD2Dyn5IPbSOkPhW65WHg/viewform?usp=publish-editor.

LLD Practice Platform

Candidate Helping Guide (Optional)

Purpose

This guide gives direction without prescribing the answer. Use it to think through the assignment; your final product and design decisions should still be your own.

1. Start With the Learner Problem

Do not begin by choosing React, Node.js, MongoDB, an LLM, or a diagram library. First understand why LLD practice is hard.

- How does a learner currently practice an LLD problem?
- How does the learner decide whether the solution is good?
- What happens when two valid designs look very different?
- What feedback would help the learner improve on the next attempt?
- What evidence should the platform retain from an attempt?

2. Do a Small Amount of Real Research

Spend roughly 2–3 hours. Look at a few LLD practice/interview tools, GitHub projects, articles, or community discussions. You do not need dozens of sources.

Look for	Questions
Practice workflow	How does the learner start, work, submit, and retry?
Submission	Is the solution text, code, UML, or something else?
Feedback	Is there a score, comments, rubric, reference solution, or AI feedback?
Learning loop	Does the product help the learner identify recurring weaknesses?
Gaps	What would you change or simplify?

3. Define a Narrow MVP

A strong 2-day MVP can be very small. For example: 3–5 problems, one practice flow, one submission model, one evaluation flow, feedback, and attempt history.

Do not build features simply because they sound impressive. Ask whether each feature improves the practice loop.

4. Think About the Submission

LLD can be expressed in several ways. Pick the smallest format that gives enough evidence of design quality.

Option	What it proves
Text design	Requirements, assumptions, classes, responsibilities, reasoning.
Code	Concrete implementation, interfaces, coupling, testability, behaviour.
Diagram	Relationships, structure, responsibilities, abstraction.
Combined	More evidence, but more implementation effort.

A good decision here is not “support everything”; it is explaining why your chosen format is sufficient for the MVP.

5. Think About Evaluation as a Rubric

Avoid treating a reference solution as the only correct answer. A useful evaluation can look at dimensions such as:

- Requirement understanding
- Class responsibilities
- Coupling / cohesion
- Encapsulation and interfaces
- Appropriate use of abstraction / patterns
- Extensibility when requirements change
- Edge cases and testability
- Quality of explanation

You can score these dimensions, provide structured comments, or combine both. The key is that feedback should point to evidence in the candidate solution.

6. Decide Where AI Helps

A practical design separates deterministic checks from judgment-heavy checks.

Good candidates for deterministic logic	Good candidates for AI
Required fields / structure	Quality of responsibilities
Compilation / tests, if code is supported	Design trade-offs
Known business rules	SOLID / abstraction analysis
Submission state transitions	Candidate explanation analysis
Idempotency / duplicate handling	Improvement suggestions

You do not need a sophisticated AI pipeline. A structured prompt + fixed rubric + stored result can be enough for the prototype.

7. Make AI Feedback More Consistent

One simple approach is to give the model a fixed rubric and require structured output.

For example:

Example evaluation shape

criterion → score → evidence → concern → suggestion → confidence

This is only a starting point. The important idea is to avoid asking the model an unconstrained question such as “Is this a good design?”

8. Keep the LLD of Your Product Meaningful

The core domain may include objects such as Problem, Attempt, Submission, Evaluation, Rubric, and Feedback—but these are examples, not a required model.

Ask of each class:

- What responsibility does it own?
- What behaviour belongs here?
- What does it depend on?
- What is likely to change?
- Do I really need this abstraction?

9. Two Simple Change Tests

Use these to challenge your design:

Change test A

Today the learner submits text. Later the platform supports a class diagram. How much of your domain model changes?

Change test B

Today feedback comes from one evaluator. Later you add a rule-based evaluator or human review. Can you add it without rewriting the practice flow?

You do not need to implement the future features. Your design just needs a sensible place for the variation.

10. Keep Scale Practical

For this assignment, scaling means showing good judgement, not producing a full HLD.

- If AI evaluation is slow, do not block the main submission request unnecessarily.
- Store the submission before evaluation starts so it is not lost on evaluator failure.
- Give evaluation a clear state such as Submitted → Evaluating → Completed / Failed.
- Avoid duplicate processing where a user retries the same request.
- If the product grows, describe the simplest component you would separate first and why.

11. Good Candidate Decisions

We would rather see:

- A simple architecture with clean domain boundaries.
- A clear reason for every important abstraction.
- A useful feedback model instead of a random “AI score”.
- A small amount of AI used thoughtfully.
- A prototype that works end-to-end.
- A clear discussion of trade-offs and limitations.

We would be less impressed by:

- Many microservices with no need.
- Lots of UI with weak practice logic.
- Design patterns added only to show pattern knowledge.
- An LLM prompt that simply asks for a 100-point score.
- Large feature lists with little working functionality.

Final reminder

Do not try to solve every possible problem. Solve one clear learner problem well, and use your design to show how the solution can evolve.